Week 13 (Deadlock Management)

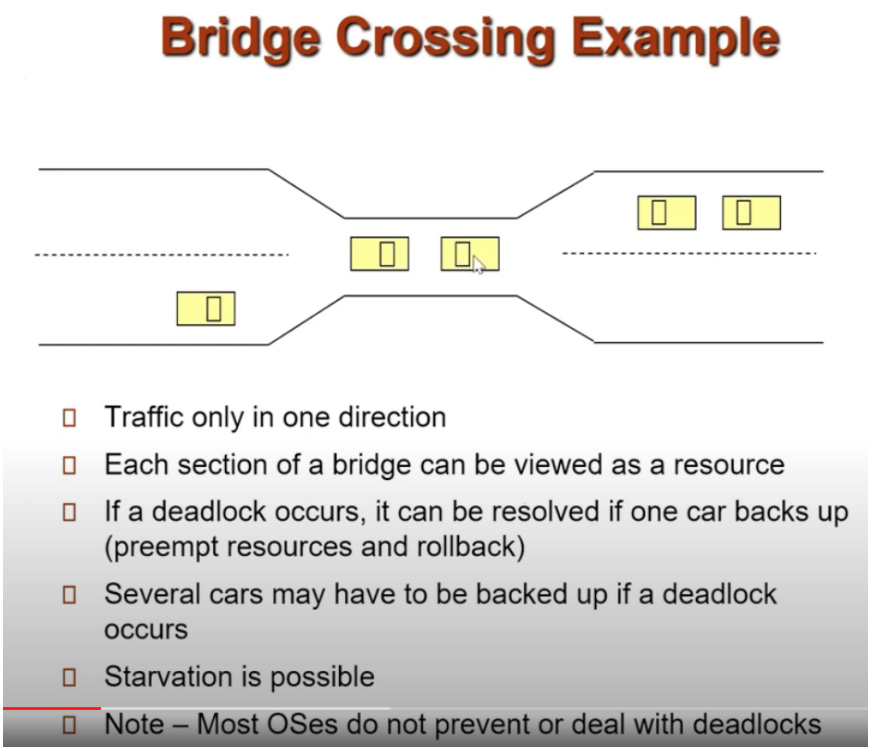
✅ Bridge Crossing Example (compare deadlock to real-world)

✅ Deadlock and Starvation

- deadlock characteristics
- Modeling deadlocks
- Method for handling deadlock

Content

✅ Bridge Crossing Example (compare deadlock to real-world)



- if noone willing to go back → deadlock

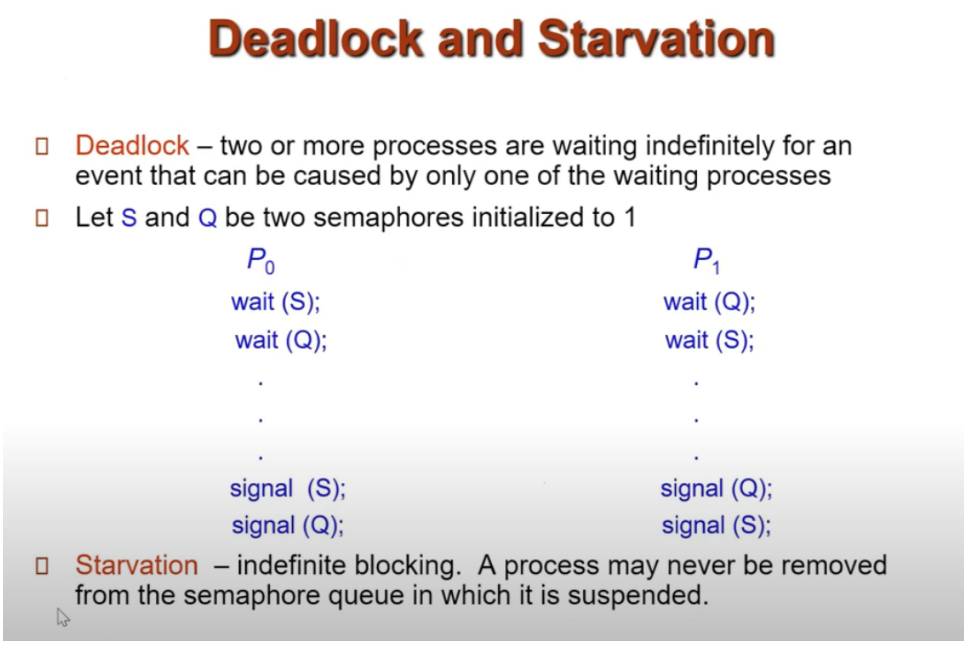
- the point is, if deadlock occur → the way to solve the solution is okay, there are car that need to back up
- and sometimes only 1 car can't solve problem, more than 1 car need to back up
- Starvation is possible?
    - what if one side agree to back up, and other side just continue without stopping

Most OS do not prevent or deal with deadlocks

- the point is os pretend that there is no deadlock in this world, and you wil learn why os does that, what this mean for us.
- if have to know that, do synchronization

✅ Deadlock and Starvation

Deadlock will occur if 4 conditions occur at the same time



- in many cases, deadlock is not easy to occur
- but sometimes, writing progrma this way can cause deadlock
    - for example S and Q are binary semaphore and all of them initiza to 1 (1 marble in ajar)
    - os may said p0 work first

        - take 1 marble form s jar
    - os may like p1 next
        - take 1 marble from q jar
    - come back to p0
        - wait forever (wait Q)

    The problem that lead to dealock → Starvation

    - the process may never work

    Same thing between deadlock and starvatio is that that process never finish

    - for deadlock it always involve mroe than 1 process → and if we not deal with it (it may extend to many process) (traffic)

📍Deadlock characteristics

(at the same time, 4 condition occur → deadlock occur)

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \ldots, P_0\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_0 is waiting for a resource that is held by P_0.
- Removal of only one condition can resolve the deadlock

1. Mutual exclusion
    - the mutual exclusion will occur when the process use a resource that allow only 1 process at a time
    - the resource can only be use by 1 process at a time.
2. Hold and wait

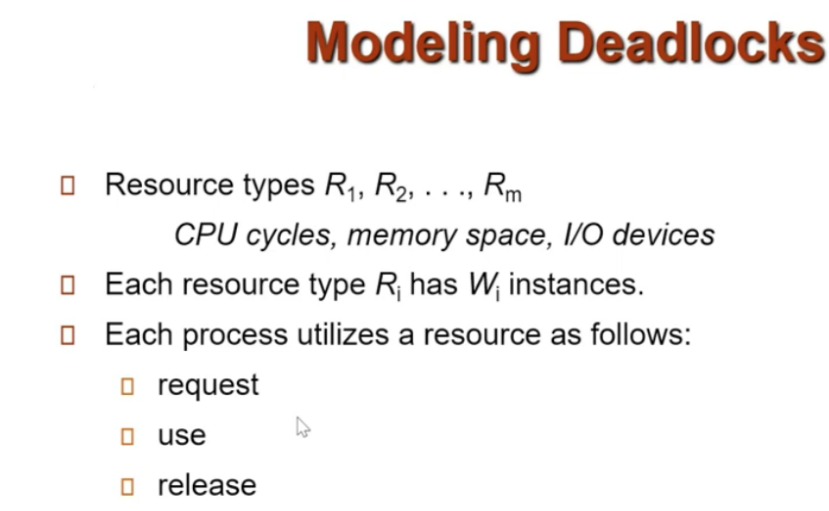
- process holding some resources and waiting for some resources that are hold by someone else.

3. No preemption
    - a resource can be taken back if the process is willing to release it itself (OS cannot do anything).
4. Circular wait
    - hold and wait must happen it circular order
        - p1 wait for p2, p2 wait for p3, p3 wait for p1
        - deadlock occur bettern p1, p2, p3

If os want to interfere, it jsut have to avoid one condition, it seem easy?

- but it not easy, and OS said no, it will not do this. Not worth it

📍Modeling Deadlock



- we want to findout if deadlock occur in our system or not, how can we modelling deadlock.
- use graph
    - node
        - R → resource
        - P → process

Resource-Allocation Graph

A set of vertices V and a set of edges E.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system.
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system.
- request edge – directed edge $P_1 \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow P_i$

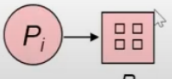
- transition from process to resource
  - request edge process request resource
- trasition from resource to process
  - resource is assign to process

Resource-Allocation Graph (Cont.)

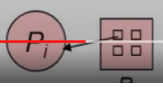
- Process

- Resource Type with 4 instances

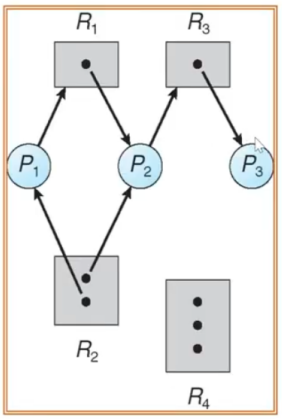
- P_i requests instance of R_j



- P_i is holding an instance of R_j



Example of a Resource Allocation Graph

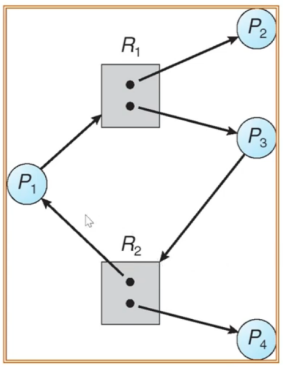


- in this system
  - 3 processes
  - 4 resources
  - R1 → 1 item
  - R2 → 2 item
    - item → number of resources
    - 2 printer
- p1 is request to r1, and hold one of r2.

we model the system with resource allocation graph

- how we model?
- we take a look, and see is there a cycle in the graph
- in this, we have cycle
  - in this system, there are deadlock

Graph With A Cycle But No Deadlock



- There cycle, but no deadlock
- we take a look at cycle, but it doesn't mean always deadlock.
  - P3 whole 1 of R1, but if P2 release, P1 can continue
  - each resource is mutual exclusion, but we have more than one resource

Conclusion

Basic Facts

- If graph contains no cycles ⇒ no deadlock.
- If graph contains a cycle ⇒
  - if only one instance per resource type, then deadlock.
  - if several instances per resource type, possibility of deadlock.

- no cycle, no circular wait → no deadlock for sure
- one instance per resource type → deadlock
- several isntance → maybe

📍Methods for Handling Deadlocks

(Assume that OS has to handle deadlock)

Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

- so there are 3 ways that os can handle deadlock (if it was)
- 2 major way

1. Ensure the system will never enter deadlock
2. Allow but recover later.
3. (choose) pretend deadlock doesn't exist.

1. Never allow
    - Deadlock prevention

Deadlock Prevention

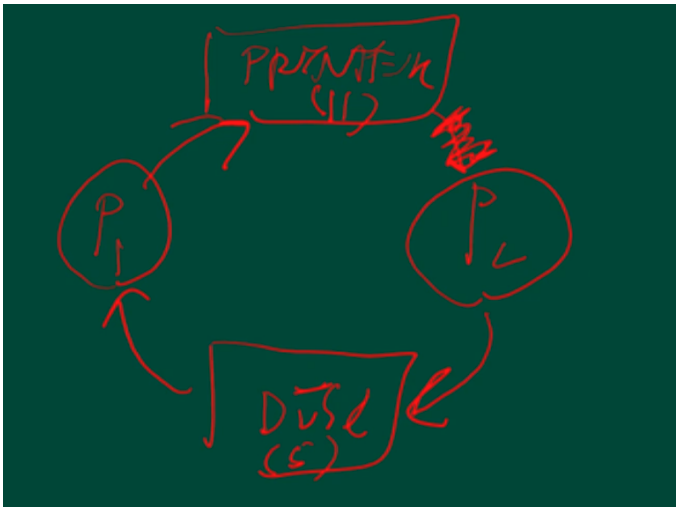
Restrain the ways request can be made

- **Mutual Exclusion** – not required for sharable resources; must hold for nonsharable resources
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
    - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none
    - Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

- **No Preemption** –
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
  - Preempted resources are added to the list of resources for which the process is waiting
  - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration

- OS need to prevent one of the condition not to occur (out of 4)
  - prevent mutual exclusion: OS can't do, nature of resource. sometimes, can't be intefere by os
  - hold and wait: os must have condition like before you start you hve to get all resource you want first. IF the process dont' get both of them just wait.
    - seems good, why os doesn't do
      - if os do, there might be some process that never get to run.
  - No preemption
    - not easy, sometime, you shouldn't get the resource back.
    - contradict with hold and wait?
  - Circular wait
    - what if os said that if you not get lower number, you can't hold the higher number
      - P2 have to be remove. (may cause starvation)



- why os doesn't do this? where number come from? maybe stat?
  - P2 is sad
  - condition that os set might not good for all process

- Deadlock avoidance
  - try to avoid the unsafe condition

Deadlock Avoidance

Requires that the system has some additional *a priori* information available.

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

Basic Facts

- If a system is in safe state ⇒ no deadlocks.
- If a system is in unsafe state ⇒ possibility of deadlock.
- Avoidance ⇒ ensure that a system will never enter an unsafe state.